DocGuard — Module 3 Pedagogical & Technical Guide

Understanding the Real Dataset Loader & Stratified Splitter (dataset_loader.py)

1. What is Module 3 Doing? (The Objective in Simple Language)

Imagine you want to train an intelligent forensic investigator (a deep neural network) to detect altered medical invoices, tax receipts, and bank statements.

To teach an AI model effectively, you need thousands of document examples divided into two categories: **Authentic (Genuine)** and **Forged (Tampered)**. But machine learning models do not learn directly from raw image files scattered in random desktop folders. Neural networks speak only one language: **multidimensional arrays of numbers (mathematical tensors)**.

Furthermore, how you feed those documents into the AI determines whether it succeeds in the real world or fails completely:
• If you feed the AI 90% authentic documents and only 10% forged documents, it becomes lazy and learns to guess 'authentic' every time.
• If you train the AI only on clean synthetic computer graphics, it gets confused when it encounters real-world documents with scanner dust, folds, and paper textures.
• If a document from the training set accidentally leaks into the validation set, the AI merely memorizes the answer like a cheating student rather than learning real forensic reasoning.

Module 3 is the intelligent logistics coordinator and data pipeline that solves these fundamental challenges.

The Core Operational Mission of Module 3:
• **Real-World Document Ingestion:** Automatically scans and organizes genuine documents into dataset/real/ and tampered documents into dataset/forged/.
• **Stratified 80/20 Partitioning:** Mathematically splits the data into 80% Training and 20% Validation while strictly guaranteeing identical class proportions in both sets.
• **Dual-Stream Tensor Packaging:** For every document, outputs synchronized RGB image tensors and in-memory ELA compression maps $(3, 224, 224)$.
• **Forensic-Safe Augmentations:** Applies subtle micro-rotations ($\pm 3^\circ$) and lighting jitter while strictly avoiding destructive horizontal flips that reverse text.
• **Hardware-Calibrated GPU Streaming:** Batches documents (8 at a time) using direct page-locked DMA memory streaming, keeping peak VRAM below 1.4 GB on the GTX 1650.

2. How Does Module 3 Achieve This? (The 4-Stage Data Pipeline)

Module 3 bridges raw image files and GPU tensor operations through a 4-stage pipeline:

• **Stage 1: Multi-Format Recursive Scanner:** Traverses root directories to ingest all supported formats (.png, .jpg, .jpeg, .tiff). Maps ground truth labels: 0 for Authentic/Real and 1 for Forged/Tampered, with automated folder verification.

• **Stage 2: Stratified 80/20 Sampling Partition:** Calculates the exact population class ratio and enforces that the 80% Training partition and 20% Validation partition contain identical class ratios. Mathematically guarantees zero data leakage between partitions (disjoint sets).

• **Stage 3: Dual-Stream PyTorch Dataset (DocGuardDataset):** Generates two parallel tensors for every document: Stream 1 (in-memory ELA compression delta map at Q=95 via Module 1) and Stream 2 (normalized RGB document image), scaled to (3, 224, 224) and standardized via ImageNet statistics.

• **Stage 4: Hardware-Calibrated DataLoader Factory:** Assembles mini-batches of 8 documents. Uses num_workers=2 to prevent Windows multiprocessing socket stalls, and enables CUDA memory pinning (pin_memory=True) for zero-copy DMA streaming to the GTX 1650 GPU.

3. Comprehensive Glossary of Technical & ML Concepts

A. Dataset Architecture & Domain Adaptation

1. The Synthetic-to-Real Domain Gap

The performance drop that occurs when a machine learning model is trained exclusively on synthetic computer-generated images and deployed on real-world scanned documents. Synthetic documents have unnaturally sharp, pixel-perfect boundaries and zero sensor noise. Authentic scanned documents contain optical lens distortion, microscopic paper grain, ink bleed, creases, and scanner illumination falloff. Training on real-world scanned documents ensures the convolutional filters learn organic forensic features rather than overfitting to synthetic artifacts.

2. Binary Ground Truth Labeling (0 vs. 1)

The mathematical encoding of document authenticity: **Y = 0** represents Authentic / Real (pristine documents issued by legitimate institutions), and **Y = 1** represents Forged / Tampered (documents containing digital text alterations, spliced figures, or erased stamps). These scalars form the target vectors for neural network loss functions.

B. Statistical Partitioning & Data Splitting

1. The 80/20 Train/Validation Split Rule

The standard practice of dividing data into a Training Set (80%), which is actively shown to the neural network during backpropagation weight updates, and a Validation Set (20%), which is held out to measure true generalization ability. The validation set acts as an honest benchmark to detect when the model begins to overfit.

2. Stratified Sampling vs. Naive Random Splitting

In naive random splitting, documents are assigned to train or validation purely by chance, which often introduces an accidental class imbalance on moderate datasets (e.g. 70% real in training, but only 30% in validation). Stratified sampling computes the exact proportion of positive samples in the full population and guarantees that both partitions maintain this exact ratio: P(Y = 1 | Train) == P(Y = 1 | Val) == P(Y = 1 | Full). This eliminates distribution drift and keeps decision thresholds calibrated.

3. Data Leakage Prevention (Disjoint Partitions)

The severe experimental error where information from the validation set contaminates the training set. If a document appears in both sets, the network merely memorizes the specific image rather than learning general forensic rules. DocGuard enforces strict disjointness: Train $\cap$ Val = $\varnothing$.

C. Forensic-Safe Data Augmentation

1. Why Standard CV Augmentations Fail on Documents

In general object recognition (e.g. cats vs. dogs), horizontal flips and heavy blurs are standard. In document forensics, flipping an image horizontally reverses text characters into illegible mirror-images, corrupting typography kernels. Similarly, heavy blurring destroys the subtle JPEG quantization equilibrium signals that Error Level Analysis (ELA) requires to spot modified digits.

2. Forensic-Safe Augmentation Policy

DocGuard applies domain-specific micro-augmentations: **Micro-Rotation ($\pm 3^\circ$)** with white paper border fill simulates slight skew from physical scanner feed rollers, and **Micro-Photometric Jitter ($\pm 5\%$)** simulates scanner lamp brightness variations without altering frequency-domain compression grids.

D. Tensor Representations & Normalization

1. Dual-Stream Tensors (RGB + ELA)

DocGuard produces two complementary tensor representations of every document:
• **Stream 1 Tensor (ELA):** Shape (3, 224, 224). Captures in-memory JPEG quantization error deltas computed at Q=95. Altered numbers glow bright white.
• **Stream 2 Tensor (RGB):** Shape (3, 224, 224). Captures raw visual layout, character kerning, and edge seams.
Feeding both streams simultaneously enables the multimodal brain to cross-reference visual alignment against compression age.

2. ImageNet Distribution Normalization (z-Score Scaling)

Pre-trained deep neural networks require input tensors normalized using standard ImageNet distribution constants: $x_{norm} = (x - \mu) / \sigma$ with $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$. This prevents gradient vanishing or explosion and ensures pre-trained weights receive inputs in their optimal numerical range.

E. Hardware Calibration & PyTorch DataLoaders

1. Batch Size Calibration for 4 GB VRAM

On the NVIDIA GeForce GTX 1650 (4,096 MiB VRAM), large batch sizes trigger CUDA Out-of-Memory crashes. Setting batch_size = 8 restricts peak activation memory to approximately 1.4 GB, leaving ample headroom for model parameters, gradients, optimizer momentum buffers, and OS display drivers.

2. Multiprocessing Workers (num_workers = 2)

On Windows 11, worker processes are created via 'spawn' rather than 'fork'. Setting num_workers = 2 on our 4-core CPU ensures that background CPU workers load and preprocess the next batch in RAM while the GPU processes the current batch, maximizing GPU utilization without socket stalls.

3. CUDA Memory Pinning (pin_memory = True)

Allocates CPU host memory as page-locked (pinned), preventing the Windows virtual memory manager from swapping it to disk. This enables direct, high-speed Direct Memory Access (DMA) transfers across the PCIe bus to GPU VRAM without CPU intervention.

Summary: How Module 3 Connects Data to Neural Streams

Pipeline Stage	Core Responsibility	Technical Mechanism	Downstream Consumer
Path Ingestion	Discovers document images	Recursive scan across 5 extensions (.png, .jpg, .tiff)	Stratified Splitter
Stratified Splitter	Guarantees class balance	Scikit-Learn train_test_split(stratify=y, test_size=0.2)	Dataset Wrappers
DocGuardDataset	Produces dual-stream tensors	Module 1 ELA engine + Torchvision transforms	PyTorch DataLoaders
Forensic Augmentation	Prevents visual overfitting	Micro-rotation (+/- 3 deg) with white paper border fill	Training Pipeline
DataLoader Factory	Hardware-calibrated streaming	Batch size 8, pin_memory=True, num_workers=2	Phases 1, 2, 3 Neural Training